

Task 1 - Deploy and Harden ledger-api

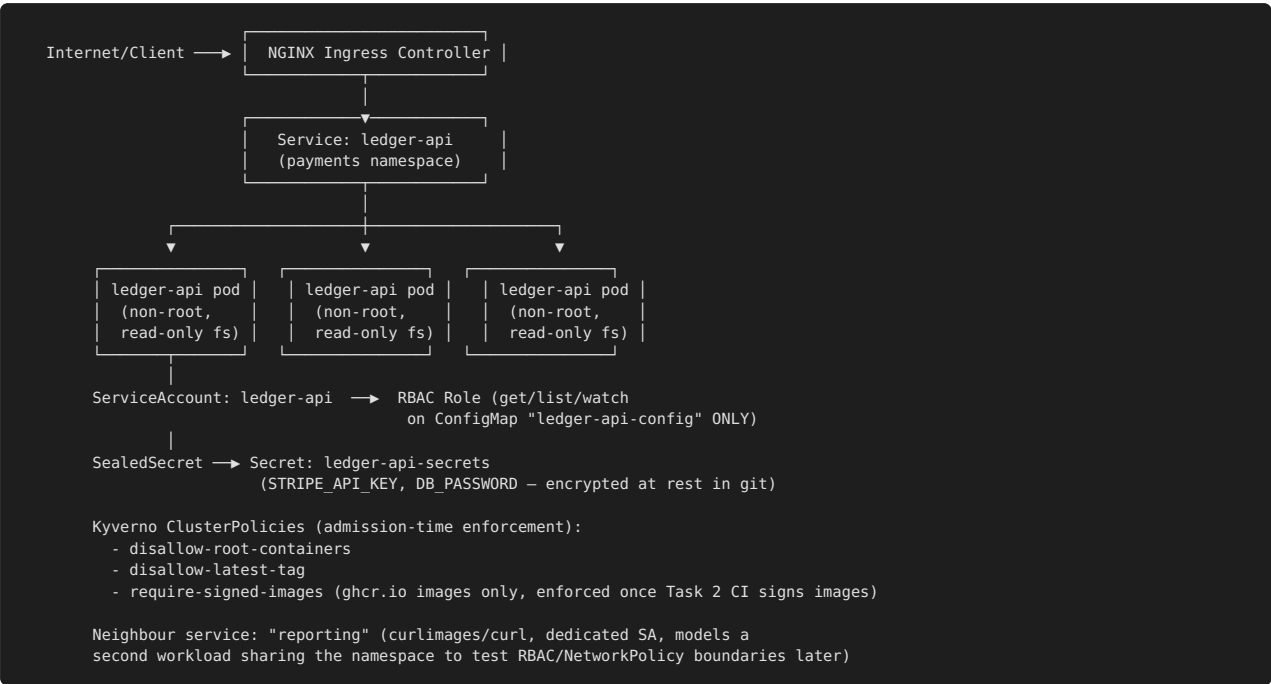
Task 1 — Deploy & Harden ledger-api

Overview

This task takes the insecure `ledger-api` starter (root container, plaintext secrets in the Deployment spec, no resource limits, no RBAC, no admission controls) and hardens it to a production-grade baseline suitable for a PCI-scoped Merchant of Record environment.

Environment: local `kind` cluster (`dodo-assignment`), Kubernetes v1.30.0, all tooling free and local — no cloud account used.

Architecture



What was insecure in the starter, and how it was fixed

Issue (starter)	Evidence	Fix
Plaintext <code>STRIPE_API_KEY</code> / <code>DB_PASSWORD</code> in <code>deployment.yaml</code> , visible via <code>kubectl describe pod</code>	<code>screenshots/01-insecure-describe-pod.png</code>	Migrated to Sealed Secrets (Bitnami). Real Secret created locally, encrypted with <code>kubeseal</code> into a <code>SealedSecret</code> that is git-safe (<code>deploy/secrets/ledger-api-sealedsecret.yaml</code>). Plaintext file is gitignored and never committed.
Container ran as root (<code>uid=0</code>)	<code>screenshots/02-root-proof.png</code> (<code>kubectl exec ... id</code>)	<code>securityContext.runAsNonRoot: true</code> , <code>runAsUser/runAsGroup: 10001</code> at pod level
No filesystem protection	—	<code>readOnlyRootFilesystem: true</code> on the container; added an <code>emptyDir</code> mount at <code>/tmp</code> for any runtime scratch space
No capability restrictions	—	<code>capabilities.drop: [ALL]</code>
No seccomp profile	—	<code>seccompProfile.type: RuntimeDefault</code>
No privilege escalation guard	—	<code>allowPrivilegeEscalation: false</code>
No resource requests/limits (QoS: BestEffort)	<code>screenshots/01-insecure-describe-pod.png</code>	<code>requests: 50m CPU / 64Mi</code> , <code>limits: 200m CPU / 128Mi</code>
No health checks	—	<code>livenessProbe</code> and <code>readinessProbe</code> on <code>GET /health</code> , 8080
Used the <code>default</code> ServiceAccount	<code>screenshots/01-insecure-describe-pod.png</code> (<code>Service Account: default</code>)	Dedicated <code>ledger-api</code> ServiceAccount, <code>automountServiceAccountToken: false</code> (app doesn't call the K8s API)

No RBAC	—	Role + RoleBinding scoped to get/list/watch on one named ConfigMap only (ledger-api-config) — verified with <code>kubectrl auth can-i</code>
No admission guardrails	—	Kyverno ClusterPolicy resources rejecting root pods, :latest tags, and (for future CI-built images) unsigned images

Files

```
deploy/
├─ namespace.yaml           # payments namespace
├─ deployment.yaml         # ledger-api ServiceAccount + hardened Deployment
├─ service.yaml            # ClusterIP service, port 8080
├─ configmap.yaml          # non-sensitive app config (LOG_LEVEL, APP_ENV)
├─ rbac.yaml               # Role + RoleBinding, least privilege
├─ ingress.yaml            # NGINX Ingress, host ledger-api.local
├─ neighbour.yaml          # "reporting" service, dedicated SA
├─ secrets/
│   └─ ledger-api-sealedsecret.yaml # encrypted secret, safe for git
└─ kyverno/
    └─ policies.yaml        # 3 ClusterPolicies (root, :latest, signed images)
```

How to reproduce

```
# 1. Cluster
kind create cluster --name dodo-assignment

# 2. Build & load image
docker build -t ledger-api:starter ./app
kind load docker-image ledger-api:starter --name dodo-assignment

# 3. Namespace, secrets controller
kubectl apply -f deploy/namespace.yaml
kubectl apply -f https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.27.1/controller.yaml
# (kubeseal CLI used locally to produce deploy/secrets/ledger-api-sealedsecret.yaml)
kubectl apply -f deploy/secrets/ledger-api-sealedsecret.yaml

# 4. Ingress controller
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml

# 5. Kyverno
kubectl create -f https://github.com/kyverno/kyverno/releases/download/v1.13.0/install.yaml
kubectl apply -f deploy/kyverno/policies.yaml

# 6. App resources
kubectl apply -f deploy/configmap.yaml
kubectl apply -f deploy/rbac.yaml
kubectl apply -f deploy/deployment.yaml
kubectl apply -f deploy/service.yaml
kubectl apply -f deploy/neighbour.yaml
kubectl apply -f deploy/ingress.yaml
```

Verification / Evidence

All commands below were run against the live cluster; screenshots in `screenshots/`.

Non-root enforced:

```
$ kubectl exec -n payments <pod> -- id
uid=10001 gid=10001 groups=10001
```

Read-only filesystem enforced:

```
$ kubectl exec -n payments <pod> -- touch /test-file
touch: cannot touch '/test-file': Read-only file system
```

RBAC — least privilege confirmed:

```
$ kubectl auth can-i get configmap/ledger-api-config -n payments --as=system:serviceaccount:payments:ledger-api
yes
$ kubectl auth can-i get secrets -n payments --as=system:serviceaccount:payments:ledger-api
no
$ kubectl auth can-i list pods -n payments --as=system:serviceaccount:payments:ledger-api
no
$ kubectl auth can-i delete configmap/ledger-api-config -n payments --as=system:serviceaccount:payments:ledger-api
no
```

Ingress routes traffic to the hardened service:

```
$ curl -H "Host: ledger-api.local" http://localhost:8080/health
{"status": "ok"}
```

Bonus — Kyverno blocks the original insecure Deployment:

```
$ kubectl apply -f insecure-deployment.yaml
Error from server: error when applying patch: ... admission webhook "validate.kyverno.svc-fail" denied the request:
resource Deployment/payments/ledger-api was blocked due to the following policies
disallow-root-containers:
  autogen-require-run-as-non-root: 'validation error: Pod and container securityContext.runAsNonRoot
  must be explicitly set to true...'
```

Design decisions & trade-offs

- **Sealed Secrets over SOPS/External Secrets:** chosen for simplicity in a fully local/free environment — no external KMS or cloud secret store required, and it integrates cleanly with GitOps (the encrypted `SealedSecret` can live safely in the same repo as the manifests).
- **RBAC scoped to a single named ConfigMap, no other permissions:** the app doesn't call the Kubernetes API at runtime, so the token is not even mounted (`automountServiceAccountToken: false`). The Role exists to demonstrate the least-privilege pattern and would be extended only if the app needed to read its own config dynamically.
- **require-signed-images Kyverno policy scoped to ghcr.io/*/ledger-api* only:** this intentionally does not block the local `ledger-api:starter` image built for this task, since signing is introduced in Task 2's CI/CD pipeline. Once Task 2 ships signed images to GHCR, this policy enforces that only those signed images are deployable.
- **Base image (python:3.6-slim) left unchanged:** Python 3.6 is EOL and ideally should be upgraded, but this is a supply-chain/dependency concern addressed more fully as part of Task 2 (SCA/CVE scanning). Flagged here as a known risk with more time.
- **/import (unsafe yaml.load) and /fetch?url= (SSRF) endpoints intentionally left as-is:** these are the target vulnerabilities for Task 4's penetration test and are out of scope for infra hardening in Task 1.

What I'd do with more time

- Add a `NetworkPolicy` default-deny + explicit allow between `ledger-api` and `reporting` now, ahead of Task 3's Istio layer, for defense-in-depth from day one.
- Enforce Pod Security Standards (`restricted`) at the namespace level as an additional guardrail alongside Kyverno.
- Add developer/operator/admin RBAC personas (bonus item) with distinct Roles.